Домашняя работа 2 — MongoDB

Разработайте каталог ShopCatalog и оформление заказа по материалам недель 4–5. Вы поработаете с документами, запросами, агрегацией, индексом и транзакцией, проведёте смену primary и объясните маршрутизацию запросов по шардам.

Ориентир трудоёмкости — 10–20 часов с готовым окружением. Максимальная оценка — 10 баллов.

Короткая инструкция

Маршрут выполнения с готовой заготовкой для `mongosh`:

1. **Сложите три файла в одну папку:** `compose.yaml`, `lab.js`, `solution.js`. Это будет папка вашего проекта.
2. **Запустите Docker и откройте терминал в этой папке.** Поднимите стенд и загрузите данные:

   ```
   docker compose up -d
   docker compose run --rm lab reset
   ```

   Получится готовый replica set из трёх узлов, 6 товаров и 35 заказов.
3. **Посмотрите данные и ожидаемые ответы:**

   ```
   docker compose run --rm lab fixtures
   ```
4. **Откройте `solution.js` и постепенно заполняйте места с `TODO`.** После каждого блока запускайте проверку:

Что реализовано	Команда
Валидатор, поиск, история и отчёт	`docker compose run --rm lab data`
Составной индекс для истории	`docker compose run --rm lab index`
Покупка с транзакцией, повтором и конфликтом	`docker compose run --rm lab purchase`
Чтение и запись при смене primary	`docker compose run --rm lab failover`

   Сценарии сами готовят данные и сохраняют результаты. При ошибке прочитайте сообщение, исправьте решение и повторите команду.
5. **Создайте README:** объясните решения по вопросам задания, заполните таблицу сравнения планов и ответьте на вопросы о шардировании. Результаты сценариев уже будут в `results/results.json`.
6. **Выполните итоговый прогон:**

   ```
   docker compose run --rm lab all
   ```

   Проверьте результаты и запишите короткое видео с показом решения. Добавьте исходники, README и результаты в репозиторий.
7. **Остановите стенд после работы:**

   ```
   docker compose down
   ```

1. С чего начать

1. **Подготовьте окружение по приложению А.** Комплект содержит `compose.yaml`, служебный скрипт `lab.js` и заготовку `solution.js`. Стенд, контрольные данные и сбор результатов уже подготовлены.
2. **Выполняйте разделы 2–7 по порядку.** В варианте с `mongosh` заполняйте функции в `solution.js`; писать генератор данных и настраивать replica set с нуля не нужно. Читать весь `lab.js` перед работой тоже не требуется.
3. **Записывайте объяснения в README.** Вопросы перечислены в соответствующих разделах; результаты сценариев сохраняет комплект.

Можно использовать другой язык с официальным драйвером MongoDB, сохранив данные, контракты и сценарии задания. Подключение описано в приложении А; окружение своего приложения вы готовите самостоятельно. Достаточно функций и запускаемых сценариев. REST API и интерфейс не требуются.

2. Данные и запросы

Каталог хранится в `products`, заказы — в `orders`. Все поля таблицы обязательны на своём уровне вложенности. Дополнительные служебные поля допустимы.

Коллекция	Поля	Типы и ограничения
`products`	`_id`	Строка — уникальный артикул
`products`	`name`, `category`	Строки — название и категория
`products`	`price`	Decimal128, неотрицательная текущая цена единицы товара
`products`	`variants`	Массив объектов с полями `size`, `stock`
`products`	`variants[].size`	Строка — размер; размеры внутри товара не повторяются
`products`	`variants[].stock`	Неотрицательное целое число
`orders`	`_id`	Строка; равна `operationId` покупки
`orders`	`userId`	Целое число — идентификатор покупателя
`orders`	`createdAt`	BSON Date — время создания заказа
`orders`	`status`	Строка: `created`, `paid` или `cancelled`
`orders`	`items`	Массив объектов с полями `sku`, `size`, `qty`, `price`
`orders`	`items[].sku`, `items[].size`	Строки — артикул из `products._id` и выбранный размер
`orders`	`items[].qty`	Положительное целое число
`orders`	`items[].price`	Decimal128, неотрицательная цена единицы на момент покупки

`[]` обозначает элемент массива и не входит в имя поля. Для целых полей допустимы BSON int32 и int64. Все цены заданы в одной валюте; товары из каталога не удаляются.

Что сделать

1. **Установите валидатор `products` в режиме `strict/error`.** Он должен проверять обязательные поля, типы, структуру элементов `variants` и неотрицательные цены и остатки. Покажите принятие корректного документа и отклонение двух документов: со строковой ценой и с отрицательным остатком. Причиной отказа должен быть именно валидатор.
2. **Реализуйте поиск товаров.** Размер и достаточный остаток должны относиться к одному варианту. Проверьте контрпример: у товара A размер L имеет остаток 0, M — 10. Поиск двух единиц L не возвращает A; после изменения остатка L на 2 — возвращает. Проверьте также другие значения всех входных параметров на общем наборе.

3. **Реализуйте историю заказов.** Проверьте состав, ограничение на число документов и порядок, в том числе при равных датах, для разных покупателей и статусов и при отсутствии совпадений.
4. **Объясните модель данных в README.** Почему позиции встроены в заказ, а история покупателя хранится отдельными документами? Какой факт представляет цена в позиции заказа?

Контракт запросов

`Product` и `Order` — полные документы из таблицы выше; возвращать только идентификаторы недостаточно.

Операция	Вход	Выход
`searchProducts`	`{category, size, qty}`	`Product[]`, подходящие товары по `_id` по возрастанию; каждый со всеми вариантами
`history`	`{userId, status}`	`Order[]`, не более 20 заказов; `createdAt` по убыванию, затем `_id` по возрастанию

При отсутствии совпадений обе операции возвращают `[]`. Входные строки непустые, `status` — из таблицы, числовые параметры — int32, `qty > 0`. Строки сравниваются точно, с учётом регистра, с `simple` collation. Обработка входов, нарушающих контракт, не требуется. Эти правила действуют и для раздела 5.

Например, поиск `{"category":"clothes","size":"L","qty":2}` возвращает B и C, а после изменения A/L — A, B и C. Контрольная история — `{"userId":2500,"status":"paid"}`. Полные данные и ожидаемые результаты доступны командой `fixtures` из приложения A.

Материал: лонгрид недели 4, §3.3 «Схема: что проверяет база, а что приложение», §3.6 «Массивы в фильтре», §4.1–4.2 о связях и исторических данных.

3. Отчёт о продажах

Нужен топ-5 товаров по выручке за переданный период `[from, to)`: начало включается, конец исключается. Даты контрольных случаев заданы в UTC.

Что сделать

1. **Напишите aggregation pipeline.** Учитывайте только `paid`, объединяйте размеры одного товара. Возвращайте `{sku, name, qty, revenue}`: суммарное количество и выручку `Σ(qty × items.price)`; `revenue` — Decimal128. Названия берите из `products` через `$lookup`. Сортируйте по выручке по убыванию, затем по артикулу по возрастанию; оставьте пять товаров.
2. **Сверьте результат с эталоном.** Проверьте все строки и их порядок, обе границы периода, исключение неоплаченных заказов и пустой период. Контрольный набор содержит больше пяти проданных товаров и равную выручку.
3. **Измените текущую цену товара в каталоге и повторите отчёт.** Историческая выручка должна остаться прежней.
4. **Покажите в README расчёт количества и выручки одного товара** по позициям исходных заказов.

Расчёт выполняется в aggregation pipeline MongoDB; расчёт циклом в приложении не выполняет условие.

Материал: лонгрид недели 4, §8.1 «От заказов к позициям и итогам по товару», §8.2 «Соединение с каталогом».

4. Индекс и план выполнения

Исследуйте запрос истории из раздела 2 на готовом наборе из 10 000 заказов. Контрольные параметры — `userId: 2500`, `status: "paid"`; запрос должен работать и с другими значениями.

Что сделать

1. **Получите план до добавления индекса.** Начальное состояние коллекции — только стандартный индекс `_id`. Используйте `explain("executionStats")`.
2. **Создайте составной индекс** для фильтра и заданной сортировки истории. Повторите тот же запрос и `explain` на неизменном наборе данных.
3. **Сравните результаты.** Сохраните полные планы и подтвердите неизменность списка `_id` и его порядка. Заполните таблицу в README.
4. **Объясните наблюдение.** Обоснуйте порядок полей индекса, опишите изменение работы запроса и одну издержку индекса. Если применяли `hint`, укажите это явно.

Вариант	nReturned	totalKeysExamined	totalDocsExamined	Есть отдельная SORT
До индекса				
После индекса				

В комплекте эти действия запускает сценарий `index` после заполнения заготовок. Время выполнения можно привести дополнительно; фиксированное ускорение не требуется.

Материал: лонгрид недели 4, §6.3 «Порядок полей в индексе и правило ESR», §6.8 «Цена индекса», раздел 7 «План запроса: explain()».

5. Оформление заказа

Покупка должна списать остатки двух разных товаров и создать один заказ в одной многодокументной транзакции и одной сессии. При отказе изменения этой покупки отменяются.

Контракт покупки

Операция `purchase` получает объект:

```
{
  "operationId": "purchase-001",
  "userId": 2500,
  "items": [
    {"sku": "A", "size": "M", "qty": 2},
    {"sku": "B", "size": "L", "qty": 1}
  ]
}
```

В `items` ровно две позиции с разными `sku`; при повторе порядок сохраняется. Цены, дату и статус определяет операция покупки, клиент их не передаёт.

Случай	Результат операции	Изменения в базе
Новая успешная покупка	Полный `Order`: `_id = operationId`, переданный `userId`, текущая дата, `status = "created"`, позиции в порядке входа с ценами покупки	Один заказ; списаны только нужные варианты и количества
Повтор подтверждённой покупки с теми же параметрами	Тот же сохранённый `Order`, включая дату и цены	Нет нового заказа и списаний
Нет товара, размера или достаточного остатка	Ошибка с `code: "ITEM_UNAVAILABLE"`	Все изменения покупки отменены, заказа нет
Прежний `operationId` с другими параметрами	Ошибка с `code: "REQUEST_CONFLICT"`	Данные не изменены

Что сделать

1. **Реализуйте покупку.** Проверка достаточного остатка должна входить в условие обновления. Сохраняйте заказ в структуре раздела 2; валидатор для `orders` не требуется. Можно использовать оболочку транзакции комплекта, разобрав её начало, commit и abort.

2. **Покажите успешную покупку.** Сравните остатки и заказы до и после, проверьте возвращённый документ.
3. **Покажите откат при нехватке второго товара.** Выведите результат первого успешного обновления внутри транзакции. После abort оба остатка должны равняться исходным, новый заказ — отсутствовать, операция — вернуть `ITEM_UNAVAILABLE`. Предварительный отказ до первого изменения не показывает откат; компенсирующие записи после покупки не заменяют транзакцию.
4. **Проверьте повтор и конфликт.** После успешной покупки повторите тот же запрос, затем измените количество с прежним `operationId`. Между этими вызовами не сбрасывайте данные. Сравнивайте значения `userId`, `sku`, `size`, `qty`, а не текст JSON или текущую цену каталога. Порядок ключей объекта не должен влиять на результат.
5. **Сохраните счётчики транзакций для успеха и отката.** Для каждого из двух опытов нужны отдельные снимки полного `serverStatus().transactions` до и после, сведения о процессе и разности счётчиков. Комплект собирает их автоматически.
6. **Объясните в README** различие атомарности и защиты от повторного бизнес-запроса, настройки транзакции и то, как ваш код приводит к commit или abort.

В `mongosh` отказ оформляется исключением с полем `code`; в другом языке — эквивалентной ошибкой. Код ошибки должен попасть в результат опыта; текст сообщения произвольный. Отсутствующий размер и SKU входят в тот же контракт, отдельная видеодемонстрация этих случаев не требуется.

Как читать счётчики

Опыт без повторных попыток	Δ totalStarted	Δ totalCommitted	Δ totalAborted
Новая успешная покупка	1	1	0
Откат после первого списания	1	0	1

Условия опыта:

- Снимки снимаются вне транзакционной сессии после подготовки данных, с одного primary и процесса: сохраняются `host`, `pid`, `localTime`, `uptime`. Между ними не запускайте другие транзакции, перезапуск или смену primary.
- Счётчики общие для процесса; они дополняют код и состояния данных, но сами по себе не доказывают атомарность покупки.
- Если драйвер повторил попытку, объясните отличие счётчиков; оно не означает автоматическую потерю баллов. Для повтора завершённой покупки нулевая разность не требуется.

Обязательный повтор покупки — последовательный, после подтверждённого завершения первой. Конкурентный нагрузочный тест и полный протокол обработки сетевых сбоев не требуются.

Материал: лонгрид недели 5, §6.1–6.4 о транзакции, повторе покупки и гарантиях; лонгрид недели 4, §3.8 «Обновление: операторы, а не значения», §3.10 «Атомарность: документ как единица». Поля счётчиков: документация MongoDB 8.0, `serverStatus`, раздел `transactions`.

6. Смена primary

Проведите отдельный опыт на готовом replica set. Заполните операции чтения и записи в заготовке; подключение, команды определения ролей и ожидание переключения предоставляет комплект.

Что сделать

1. **Дождитесь готовности трёх узлов** и сохраните их роли.
2. **Запишите контрольный документ** через клиент replica set с `writeConcern: {w: "majority", j: true}`. Дождитесь подтверждения.
3. **Вызовите `rs.stepDown()`** или соответствующую команду драйвера и дождитесь нового primary.

4. **Тем же экземпляром клиента прочитайте контрольный документ и выполните новую запись с тем же write concern.** Не меняйте вручную адрес подключения. Сохраните новые роли и результаты операций. Кратковременную ошибку можно обработать повтором после готовности primary; сохранять одно физическое соединение не требуется.
5. **Объясните в README**, как клиент находит новый primary и почему majority-подтверждение не гарантирует немедленно актуальное чтение с любой secondary.

Фиксированное время переключения, разделение сети и потерю узлов проверять не нужно.

Материал: лонгрид недели 5, разделы 3–5 о write concern, выборах и чтении. Команда переключения: документация MongoDB 8.0, `rs.stepDown()`.

7. Шардирование

Ответьте в README без развёртывания шардированного кластера. Коллекция `orders` распределена по ключу `{userId: 1}`; карта неизменна, на обоих шардах есть документы.

Диапазон userId	Шард
Меньше 2000	A
От 2000 включительно до 5000 исключительно	B
От 5000 включительно	A

1. Какие шарды опросит `mongos` для истории `{userId: 2500}`, а какие — для общего отчёта по `{status: "paid"}`? Обоснуйте оба ответа.
2. Изменится ли число участников второго запроса после создания индекса `{status: 1}` внутри шардов? Какую работу этот индекс способен изменить?

Материал: лонгрид недели 5, §7.2–7.4 о распределении документов, маршрутизации и индексах.

8. Результаты и демонстрация

1. **Сохраните фактические результаты разделов 2–6 в `results/`.** Нужны ответы запросов и отчёта, проверка валидатора, полные планы, состояния покупок и ошибки, счётчики транзакций, роли и операции при переключении. Комплект сохраняет их автоматически в Extended JSON, сохраняя BSON-типы и точность значений.
2. **Добавьте в README объяснения из разделов 2–7 и команды воспроизведения.** Укажите изменения окружения и использованные примеры курса. Отдельный отчёт не нужен.
3. **В демонстрации покажите все обязательные сценарии разделов 2–6.** Кратко пройдите результаты запросов и отчёта; подробнее объясните индекс, покупку с откатом и повтором, смену primary. Покажите соответствующий код и фактический вывод. Сохранённый вывод обозначьте как сохранённый и назовите команду получения.

Ориентир видео — 5–8 минут, без оценки монтажа и длительности. Окружение можно подготовить заранее. Надпись «все проверки пройдены» не заменяет результаты операций; ответы о шардировании достаточно оставить в README.

9. Оценивание

Максимум — **10 баллов**. Баллы складываются из выполненных частей каждого раздела. Для зачёта части нужно выполнить её требования, включая контрольные случаи и объяснения; каждая часть стоимостью 0,5 балла засчитывается целиком или не засчитывается.

Критерий	Баллы	Из чего складывается оценка
Данные, модель и запросы — раздел 2	**1,5**	Валидатор и объяснение модели — 0,5; поиск — 0,5; история заказов — 0,5
Отчёт о продажах — раздел 3	**2**	Отбор заказов — 0,5; расчёт количества и выручки — 0,5; `$lookup` и топ-5 — 0,5; сверка, расчёт одного товара и опыт с ценой — 0,5

Критерий	Баллы	Из чего складывается оценка
Индекс и план — раздел 4	**1,5**	Составной индекс — 0,5; сравнение планов и выдачи — 0,5; объяснение результата и издержки индекса — 0,5
Транзакционная покупка — раздел 5	**2,5**	Успешная покупка и объяснение транзакции — 0,5; откат — 1; повтор и объяснение его отличия от атомарности — 0,5; конфликт параметров — 0,5
Смена primary — раздел 6	**1**	Опыт с ролями, чтением и записями через тот же клиент — 0,5; объяснение поведения клиента и чтения с secondary — 0,5
Шардирование — раздел 7	**0,5**	Верные и обоснованные ответы на оба вопроса — 0,5
Воспроизводимость и результаты — раздел 8, приложение A	**1**	Настройки и команды позволяют повторить опыты — 0,5; сохранён полный фактический вывод, включая снимки транзакций, — 0,5
Итого	**10**	

За откат начисляются независимо **0,5 за сохранность обоих остатков** и **0,5 за отсутствие заказа с ошибкой `ITEM_UNAVAILABLE`**. Успех и обе части отката требуют одной реальной многодокументной транзакции и одной сессии. Для отката обязательно показать первое успешное изменение внутри неё и последующий abort.

Ошибки одной части не обнуляют остальные. Отсутствие снимков счётчиков учитывается в комплектности результатов, а не как отдельный штраф за покупку. Сохранённый неверный результат оценивается по содержательному критерию, а не как отсутствие результата. При снижении оценки проверяющий указывает незачтённую часть и причину; дополнительных штрафов нет.

Проверка опирается на код, README, результаты и демонстрацию. Параметры могут быть изменены в пределах контрактов. Самопроверка помогает найти ошибки и не заменяет объяснений; отдельный проверяющий скрипт от студента не требуется.

Приложение A. Подготовка и команды

Подготовка

1. **Установите и запустите Docker с Compose v2.** Для трёх узлов практический ориентир — 4 ГБ памяти, доступной Docker. При первом запуске нужен интернет для скачивания образа. MongoDB и языковые среды на хосте для варианта `mongosh` не нужны.
2. **Положите три файла комплекта в корень своего проекта:** `compose.yaml`, `lab.js`, `solution.js`. Откройте терминал в этой папке.
3. **Поднимите окружение и загрузите контрольные данные:**

```
docker compose up -d
docker compose run --rm lab reset
```

4. **Посмотрите данные и ожидаемые ответы**, затем заполняйте `solution.js`:

```
docker compose run --rm lab fixtures
```

Работа со стендом

Команда после `docker compose run --rm lab`	Действие
`data`	Проверить валидатор, поиск, историю и отчёт
`index`	Сравнить планы на 10 000 заказов
`purchase`	Проверить покупку, откат, повтор и конфликт; снять счётчики
`failover`	Выполнить опыт смены primary
`all`	Последовательно выполнить все четыре сценария

Команда после `docker compose run --rm lab`	Действие
`reset`	Восстановить 6 товаров и 35 заказов; `reset index` — набор из 10 000 заказов. Прежние результаты проверок очищаются
`fixtures`	Показать контрольные данные, параметры и эталоны
`shell`	Открыть интерактивный `mongosh` для своих запросов
`help`	Показать справку

Например: `docker compose run --rm lab purchase`. Каждый сценарий сам готовит своё исходное состояние; не запускайте их одновременно и не меняйте контрольный набор. До заполнения соответствующих функций сообщение `TODO` ожидаемо. Для полного прогона используйте `all`.

Результаты собраны в `results/results.json`, отдельно по сценариям, включая полные планы и снимки транзакций. Проверьте дату и успешность последнего запуска. Для остановки выполните `docker compose down`. Чтобы удалить также учебные данные в Docker и начать заново, выполните `docker compose down -v`, затем повторите две команды подготовки. Файл результатов на хосте при этом остаётся.

Настройки

Параметр	Значение
MongoDB	Образ `mongo:8.0.30`, `mongosh` внутри образа
Топология	Replica set `hw2rs`, три узла с данными: `m1:27017`, `m2:27017`, `m3:27017`
Базы	`hw2` для запросов и покупки, `hw2_failover` для смены primary
Доступ	Внутренняя Docker-сеть, без аутентификации и TLS; порты на хост не публикуются
Чтение и транзакция	`readPreference: primary`, `readConcern: snapshot` для транзакции
Подтверждение записи	`{w: "majority", j: true, wtimeout: 10000}`

Для другого языка запускайте приложение в той же Docker-сети, например отдельным сервисом в `compose.yaml`. Используйте URI replica set из справки `help`; все три адреса должны быть доступны клиенту. Команды `reset` и `reset index` готовят данные без заполнения `solution.js`. Сохраните те же сценарии и результаты, команды своего приложения укажите в README. Прохождение встроенного запуска JS-заготовки для этого варианта не требуется.